

UniAgent: An End-to-End Decentralized Framework for Autonomous Agent Discovery, Delegation, and Payment

Ryoto Taga
ryoto.taga12@gmail.com
Accenture
Tokyo, Japan

M. Fahim Ferdous Khan
khan@iniad.org
Faculty of Information Networking for Innovation and
Design (INIAD)
Toyo University
Tokyo, Japan

Abstract

Large language model (LLM) agents are increasingly able to act on behalf of users, but the infrastructure that lets them find, hire, and pay one another across organizational boundaries is still fragmented. Open protocols already exist for the individual pieces: the Agent2Agent (A2A) protocol standardizes agent-to-agent communication, x402 turns the HTTP 402 status code into a micropayment channel, and ERC-8004 records agent identity on chain. However, these protocols have so far been demonstrated only in isolation or as one-to-one proofs of concept, and no end-to-end service architecture ties them together so that a single natural-language request is autonomously decomposed into agent discovery, delegation, and payment. In this paper we present *UniAgent*, a decentralized agent marketplace that integrates A2A, x402, and ERC-8004 under an LLM-based orchestration layer. A three-layer design separates a *marketplace layer* (an on-chain ERC-8004 registry kept in sync with an off-chain cache through blockchain event webhooks), an *orchestration layer* (a ReAct agent that discovers, inspects, and invokes external agents through three tools), and a *payment layer* (x402 with delegated signing). We describe a working prototype deployed on the Base Sepolia testnet and report an end-to-end evaluation over 20 flight- and hotel-search tasks. UniAgent shows that open communication, on-chain identity, and protocol-native micropayments can be combined into a coherent, vendor-neutral marketplace in which user tasks are carried out across separately deployed A2A services.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Information systems** → *Web services*; • **Software and its engineering** → *Distributed systems organizing principles*.

Keywords

AI agents, agent marketplace, A2A protocol, x402, ERC-8004, micropayments, on-chain identity, LLM orchestration, ReAct

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCCM '26, Tokyo, Japan

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2439-8
<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Ryoto Taga and M. Fahim Ferdous Khan. 2026. UniAgent: An End-to-End Decentralized Framework for Autonomous Agent Discovery, Delegation, and Payment. In *Proceedings of The 14th International Conference on Computer and Communications Management (ICCCM 2026) (ICCCM '26)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Problem setting. LLM agents can increasingly plan, call tools, and complete multi-step tasks for users. A natural next step is cross-organizational delegation: for example, a travel assistant should be able to discover, hire, and pay specialized flight and hotel agents operated by different providers. This requires open mechanisms for communication, identity and discovery, and payment across trust boundaries.

Gap. Relevant building blocks now exist: A2A standardizes agent-to-agent communication [8], ERC-8004 provides on-chain agent identity [6], and x402 attaches micropayments to HTTP requests [5]. Yet these protocols do not by themselves provide an end-to-end marketplace. A2A leaves discovery to applications; Vaziry et al. [9] demonstrate protocol feasibility only in fixed one-to-one interactions; and centralized agent stores such as the GPT Store [3] provide smooth experiences only inside platform-specific registration, discovery, and billing systems. Moreover, combining these protocols is not straightforward: A2A deliberately excludes discovery from its scope, so a registry mechanism must be added externally; ERC-8004 provides identity but defines no interface for capability-based search; and x402 settles payments independently of agent identity, requiring an integration point that binds payment authorization to a verified registry entry. No existing design addresses these gaps together.

Research question. This motivates our research question:

How can open agent communication, on-chain identity, and protocol-level micropayments be integrated into a cohesive marketplace architecture with LLM-based orchestration, so that a user's high-level task is autonomously decomposed into agent discovery, delegation, and payment across organizational boundaries?

UniAgent overview. We answer this question with *UniAgent*, a decentralized agent marketplace summarized in Figure 1. UniAgent re-implements the registration, discovery, and payment functions of a closed platform using A2A, ERC-8004, and x402, and places an LLM ReAct orchestrator [11] above them to analyze the user's

task, discover candidate agents, inspect their specifications, select providers, and execute paid calls.

Contributions. This paper makes three contributions:

- (1) **An integrated marketplace architecture** that unifies A2A, x402, and ERC-8004 behind three loosely coupled layers: marketplace, orchestration, and payment.
- (2) **An autonomous LLM pipeline** in which a ReAct agent drives task analysis, discovery, specification inspection, and paid execution through `discover_agents`, `fetch_agent_spec`, and `execute_and_evaluate_agent`.
- (3) **A working prototype and end-to-end demonstration**¹ on Base Sepolia, exercising the full flow over external flight- and hotel-search services and reporting end-to-end success rate.

2 Background

UniAgent builds on four open technologies. The *Agent2Agent* (A2A) protocol [8] standardizes inter-agent communication: each agent publishes an Agent Card at a well-known URL describing its capabilities and endpoint, and clients exchange tasks over JSON-RPC. A2A deliberately leaves agent *discovery* outside its scope. The *x402* protocol [5] revives the HTTP 402 status code for micropayments: a server responds with payment requirements, the client signs a stablecoin transfer, and a facilitator settles the transaction on chain. *ERC-8004* [6] provides an on-chain agent identity registry built on ERC-721, assigning each agent a token whose metadata is stored off chain; because the registry is a public contract, any client can enumerate agents without trusting a central catalog. Finally, the *ReAct* pattern [11] interleaves LLM reasoning and tool calling, which suits orchestration where the set of agents and prices must be explored at run time.

3 Related Work

Agent communication protocols. Several protocols now compete to standardize how agents interact. Ehtesham et al. [1] survey this space and compare the Model Context Protocol (MCP), which connects a single agent to tools and data sources, with peer-to-peer protocols such as A2A and the Agent Network Protocol (ANP). UniAgent adopts A2A for inter-agent communication because it is designed for cross-organization delegation. None of these communication protocols specifies discovery or payment, which is precisely what UniAgent’s marketplace and payment layers add.

Decentralized agent identity and closest prior work. Vaziry et al. [9] represent each agent’s card as an individual smart contract and show that A2A, x402, and on-chain identity can interoperate. UniAgent instead consolidates identities into a single standardized ERC-8004 registry [6] rather than per-agent contracts, and—more importantly—builds a full marketplace and orchestration layer on top, as detailed below.

Vaziry et al. establish the technical *feasibility* of the protocol combination through one-to-one agent interactions. UniAgent treats those protocols as building blocks for a marketplace architecture: a standardized ERC-8004 registry and synchronized cache support

discovery, while an LLM orchestrator starts from a user’s natural-language task, selects providers, pays them over x402, and returns an integrated result. Thus, Vaziry et al. demonstrate protocol feasibility, whereas UniAgent contributes the end-to-end service layer above those protocols.

LLM multi-agent orchestration. AutoGen [10] shows how multiple LLM-based agents can coordinate through conversation, tool use, and optional human input within an application. UniAgent addresses a different layer: it treats the marketplace itself—open registration, cross-vendor discovery, and protocol-native payment—as the object of orchestration.

Centralized marketplaces. Centralized agent stores such as the OpenAI GPT Store [3] and Oracle AI Agent Marketplace [4] show the demand for integrated agent marketplaces. Oracle’s adoption of A2A also suggests that agent-interface standards are becoming commercially relevant. However, in these systems, registration, discovery, deployment, and billing remain confined to vendor platforms and rely on platform-managed payment rather than protocol-native micropayments. Rothschild et al. [7] frame the broader tension as a choice between closed “agentic walled gardens” and an open “web of agents.” UniAgent targets the latter by combining open protocols for communication, identity, discovery, and payment.

4 System Architecture

UniAgent is organized into three layers, shown together in Figure 1: a *marketplace layer* that maintains the catalog of available agents, an *orchestration layer* that turns a user task into a sequence of agent invocations, and a *payment layer* that settles each invocation. The layers are deliberately decoupled—the orchestrator does not need to know how identity is stored, and the payment layer does not need to know why an agent was selected—so that each can evolve independently. We describe each layer in turn and then explain how they combine into an end-to-end flow.

4.1 Marketplace Layer

The marketplace layer answers a single question: which agents exist and what can they do? Its authoritative source is the on-chain ERC-8004 AgentIdentityRegistry. An agent developer deploys an A2A agent, uploads its Agent Card to IPFS, and registers it in the registry with the IPFS content identifier as the token’s metadata URI, optionally staking a deposit. Reading the chain on every request would be slow and costly, so UniAgent maintains an off-chain PostgreSQL cache synchronized through blockchain event webhooks: registration and staking events trigger a webhook that fetches the metadata from IPFS and updates the cache. The marketplace UI reads from this cache, while the chain remains the single source of truth.

4.2 Orchestration Layer

The orchestration layer is a ReAct agent [11] built with LangChain and driven by a commercial LLM. It receives the user’s natural-language task and operates the marketplace through three tools:

- `discover_agents(category, query)` returns a short list of candidate agents for a given need, drawn from the marketplace cache (Section 4.1).

¹Source code: <https://github.com/toryoto/UniAgent>; live demo: <https://uni-agent-web.vercel.app/>.

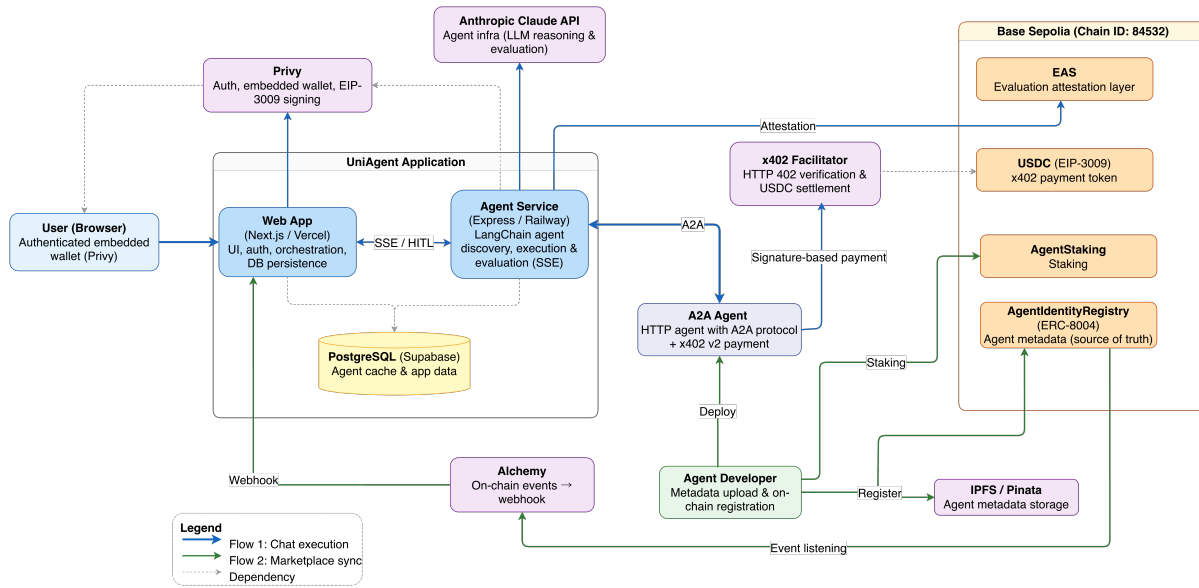


Figure 1: Overview of the UniAgent architecture. A user issues a natural-language task through the web application; an LLM-based agent service discovers candidate agents from an ERC-8004 registry (cached in PostgreSQL via Alchemy webhooks), invokes the selected external A2A agents, and settles payment over x402 using USDC on Base Sepolia.

- `fetch_agent_spec(agentId)` retrieves the candidate's Agent Card from its well-known URL and, when available, its OpenAPI specification. This lookup is free of charge and lets the LLM inspect skills, input modes, endpoint details, and x402 pricing before committing to a paid call.
- `execute_and_evaluate_agent(agentId, category, input)` invokes the chosen agent over A2A, settles payment over x402 (Section 4.3), verifies the resulting on-chain transaction, and records a quality attestation through EAS.

Orchestration loop. Given a task such as planning a trip, the ReAct agent first decomposes it into sub-tasks (e.g., a flight search and a hotel search). For each sub-task it calls `discover_agents` to obtain up to three ranked candidates, then typically calls `fetch_agent_spec` on the most promising ones to compare capabilities and prices. After selecting an agent, it calls `execute_and_evaluate_agent`, observes the result, and repeats the loop until the overall task is complete, at which point it composes a final answer. Crucially, the model is never given a fixed plan or a hard-coded list of agents—both the decomposition and the choice of providers emerge from the ReAct loop at run time.

Discovery ranking. The ranking logic is an implementation detail rather than a contribution of this paper. In the prototype, `discover_agents` returns up to three candidates from the marketplace cache, prioritizing agents with higher observed reliability and quality, larger deposits, and recent registration. This keeps the LLM’s choice set small while preserving a basic signal of provider trustworthiness.

4.3 Payment Layer

The payment layer settles each paid invocation using the x402 verify-and-settle pattern [5]. When the orchestrator calls an external A2A agent, the agent responds with HTTP 402 and a PAYMENT-REQUIRED header specifying the price and recipient. The agent service obtains a delegated payment signature from a Privy wallet without exposing the user’s private key, and retries the request with the signed payload. The external agent verifies and settles the payment through an x402 facilitator, which submits the USDC transfer on chain. The main UniAgent extension to the standard x402 pattern is *delegated signing* through Privy, which makes payments gasless and non-custodial. After a successful execution, the system records an off-chain quality and reliability attestation using the Ethereum Attestation Service (EAS) [2]; these attestations can be used as reputation signals in future discovery and ranking.

4.4 Integration Design

After the user authenticates with Privy and submits a task, the web application opens a server-sent events (SSE) stream and forwards the task to the orchestrator. The orchestrator runs the ReAct loop—discovering agents (marketplace layer), inspecting their Agent Cards, and executing the chosen agent with x402 payment (payment layer). Intermediate events—reasoning steps, tool calls, payments, and partial results—are streamed back to the UI over SSE.

The value of the design comes from the combination rather than any single protocol. A2A makes agents callable across organizations; ERC-8004 makes them discoverable without a central directory; x402 makes them payable with a single signed request; and the orchestration layer ties these together so that a user expresses

a goal in natural language and the system carries out discovery, delegation, and payment automatically.

5 Implementation

We implemented UniAgent as a TypeScript monorepo; the source code is available at <https://github.com/toryoto/UniAgent> and the live deployment at <https://uni-agent-web.vercel.app/>. The web application uses Next.js and React, with Privy for social login and delegated wallets. The orchestration service runs separately on LangChain with a commercial LLM, exposing the three tools described in Section 4.2 and streaming results over SSE. The marketplace cache is stored in PostgreSQL and accessed through an ORM, with all database access confined to a dedicated data-access layer. Smart contracts are written in Solidity and developed with Hardhat. Payments use the x402 protocol with USDC on the Base Sepolia testnet.

Deployment. The prototype runs on the Base Sepolia testnet (chain ID 84532), with an ERC-8004 AgentIdentityRegistry for agent identity and an AgentStaking contract for deposits. An EAS schema records per-interaction quality and reliability attestations.

Demonstration scenario. For the prototype we focus on a travel-planning use case with two agent categories, *flight* and *hotel*, served as external A2A services. A request such as “plan a three-day trip to Paris” is decomposed by the orchestrator into a flight sub-task and a hotel sub-task, each resolved through discovery, specification inspection, and paid execution, and the results are combined into a single itinerary. The evaluated marketplace is intentionally small but heterogeneous. In each category we deploy one A2A agent backed by a real travel-search API and ten dummy A2A agents to exercise discovery, ranking, and provider-quality variation. Thus the evaluation measures protocol-level cross-service integration rather than production-scale provider coverage.

The web interface includes a chat view for natural-language task submission and a marketplace view for browsing registered agents by category and price. During execution, the chat view streams tool calls, candidate recommendations, human-in-the-loop approval prompts, payment events, and final results over SSE.

6 Evaluation

Our evaluation asks whether UniAgent can carry a realistic user task through discovery, delegation, and payment end to end. We report 20 completed runs—ten single-agent hotel tasks (S1) and ten multi-agent flight-plus-hotel tasks (S2)—on Base Sepolia with flight and hotel A2A services (one API-backed agent and ten dummy agents per category). Each scenario class uses ten natural-language prompts with varied destinations, dates, and travel intent.

6.1 Experimental Setup

All runs use the same orchestrator prompt, which defines discovery, specification lookup, user approval, paid execution, and final reporting without encoding fixed answers or success labels. Before paid execution, the chat view presents candidate providers and requires explicit user approval; we therefore measure an approval-mediated end-to-end workflow rather than autonomous provider selection alone. A run succeeds only if it completes discovery, specification

lookup, candidate presentation, user approval, paid execution, x402 settlement, and final reporting. We record transaction hashes and execution logs, then manually label protocol completion and task success separately.

6.2 Scenarios and End-to-End Success Rate

We use two scenario classes. S1 tests a single paid agent call using hotel-search tasks. A representative prompt is: “Search for a hotel in Tokyo for 2 adults from July 24 to July 26, 2026.” A successful S1 run requires that one hotel agent be discovered, approved, paid, executed, and returned in the final response with hotel results.

S2 tests multi-agent composition. The orchestrator must decompose a travel-planning request into flight and hotel sub-tasks, discover and inspect providers for each role, execute both providers, and integrate their results. A representative prompt is: “Plan a business trip from Tokyo to London from September 3 to September 5, 2026, including flights and a business hotel.”

Table 1 reports the observed completion rates for these scenario classes.

Table 1: Observed end-to-end results by scenario.

Scenario	Type	Runs	Protocol	Task
S1	single agent	10	10/10	10/10
S2	multi-agent	10	10/10	8/10

Representative case. S2-02 illustrates the full multi-provider path. For a Tokyo–London business-trip request, the orchestrator decomposes the task into flight and hotel searches, discovers and inspects providers for each role, presents priced candidates for approval, and then executes two paid A2A calls. A successful run requires two x402 transaction hashes, two agent results, and a final response that integrates the flight and hotel options.

The table separates protocol completion from task-level success. Protocol completion means that UniAgent reached paid execution, settled the x402 payment, returned transaction hashes, and produced a final report. Task success additionally requires the external provider results to match the requested city, dates, and role. All S1 and S2 runs completed the protocol path. S1 also achieved 10/10 task success. S2 achieved 8/10 task success: in S2-03, the hotel provider returned mainly Johor Bahru results for a Singapore family trip, and in S2-07 the flight provider returned a Tokyo–London route for a Tokyo–New York request. In both cases, UniAgent still completed discovery, payment, execution, and final reporting, and the orchestrator surfaced the mismatch rather than presenting the output as fully correct. Minor provider-quality issues, such as duplicate flight listings or mixed nearby-city hotel results, were recorded separately from task failures.

Table 2 compares UniAgent feature-by-feature with the protocol-level study of Vaziry et al. [9] and with centralized platforms such as the GPT Store [3] and Oracle AI Agent Marketplace [4].

6.3 Comparison with Existing Approaches

Oracle’s adoption of A2A illustrates that agent-interface standards are becoming commercially relevant, but the benefit remains limited if registration, discovery, billing, and deployment stay inside a

Table 2: Feature comparison.

Feature	Vaziry et al.	UniAgent	GPT Store	Oracle Marketplace
Registration	Custom contract	ERC-8004	Platform-specific	Validated templates
Discovery	Proposed, not integrated	Registry + cache	In-platform search	Within Fusion Apps
Payment	x402 PoC	x402 + delegated signing	Platform billing	Platform billing
Orchestration	None	LLM ReAct	Platform-dependent	App-embedded
End-to-end flow	No	Yes	Yes (closed)	Yes (closed)
Vendor-neutral	Yes	Yes	No	No

single vendor’s application layer. UniAgent matches the end-to-end convenience of centralized platforms while keeping registration, discovery, and payment open and vendor-neutral, and it adds the orchestration layer that the protocol-level study lacks. The centralized platforms achieve an end-to-end flow only within closed environments, whereas UniAgent does so across separately deployed A2A services.

The evaluation results should be read in this context. UniAgent’s strongest measured result is protocol completion: every observed S1 and S2 run reached paid execution and returned verifiable x402 transaction hashes. The task-level failures in S2 were not caused by the marketplace, payment, or A2A integration layers, but by provider output quality. This distinction is important for an open marketplace architecture. The framework can make discovery, payment, execution, and reporting auditable across provider boundaries, while still exposing quality differences among separately deployed services. In contrast, centralized marketplaces can hide those boundaries behind a platform-specific runtime, but they also bind discovery, billing, and orchestration to a single vendor.

7 Discussion

The prototype shows that an open, integrated marketplace is feasible: open registration, discovery, communication, and payment protocols can be composed into an end-to-end marketplace when connected by an LLM orchestrator. Its main limitation is scale: the system currently runs on a testnet, uses two travel-related agent categories, and evaluates a small population of external A2A services. The discovery ranking and EAS-based reputation feedback are implemented as design elements but have not yet been validated under large-scale or adversarial marketplace conditions.

End-to-end quality also depends on provider output quality and the orchestrator’s selection accuracy. In our evaluation, protocol completion remained high even when task-level success was reduced by provider mismatches, suggesting that future work should focus on failure recovery, stronger reputation signals, mainnet hardening, and larger multi-agent workflows.

8 Conclusion

We presented UniAgent, a decentralized agent marketplace that integrates the A2A communication protocol, x402 micropayments, and ERC-8004 on-chain identity under an LLM-based orchestration layer. The main contribution of our architecture is the separation and integration of three responsibilities that are usually handled by closed platforms: a marketplace layer for open agent registration and discovery, an orchestration layer that turns a user request into

agent selection and delegation, and a payment layer that settles cross-provider execution through protocol-native micropayments. Whereas prior work demonstrated these protocols in isolation or as one-to-one interactions, UniAgent shows how they can be composed into a coherent, vendor-neutral framework in which a single natural-language request is autonomously decomposed into agent discovery, delegation, and payment. A working prototype on Base Sepolia demonstrates the full flow over separately deployed flight- and hotel-search services, showing that end-to-end convenience need not depend on a centralized agent platform. We believe this integrated design is a step toward an open “web of agents” in which users delegate tasks across organizational boundaries as easily as they browse the web today.

References

- [1] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). <https://arxiv.org/abs/2505.02279>. arXiv:2505.02279. Accessed: 2026-05-31.
- [2] Ethereum Attestation Service. 2024. Ethereum Attestation Service (EAS) Documentation. <https://docs.attest.org/>. Accessed: 2026-05-31.
- [3] OpenAI. 2024. Introducing the GPT Store. <https://openai.com/index/introducing-the-gpt-store/>. Accessed: 2026-05-31.
- [4] Oracle. 2025. Oracle Launches Fusion Applications AI Agent Marketplace to Accelerate Enterprise AI Adoption. <https://www.oracle.com/news/announcement/ai-world-oracle-launches-fusion-applications-ai-agent-marketplace-to-accelerate-enterprise-ai-adoption-2025-10-15/>. Press release. Accessed: 2026-06-10.
- [5] Erik Reppel, Ronnie Caspers, Kevin Leffew, Danny Organ, Dan Kim, and Nemil Dalal. 2025. x402: An Open Standard for Internet-Native Payments. <https://www.x402.org/x402-whitepaper.pdf>. Coinbase Developer Platform whitepaper. Accessed: 2026-05-31.
- [6] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. 2025. ERC-8004: Trustless Agents. <https://eips.ethereum.org/EIPS/eip-8004>. Ethereum Improvement Proposals, no. 8004, August 2025. Accessed: 2026-05-31.
- [7] David M. Rothschild, Markus Mobius, Jake M. Hofman, Eleanor W. Dillon, Daniel G. Goldstein, Nicole Immorlica, Sonia Jaffe, Brendan Lucier, Aleksanders Slivkins, and Matthew Vogel. 2026. The Agentic Economy. *Commun. ACM* 69, 2 (Feb. 2026), 39–42. doi:10.1145/3747175
- [8] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. 2025. Agent2Agent (A2A) Protocol. <https://a2a-protocol.org/>. Specification, Google. Accessed: 2026-05-31.
- [9] Awid Vaziry, Sandro Rodriguez Garzon, and Axel Küpper. 2025. Towards Multi-Agent Economies: Enhancing the A2A Protocol with Ledger-Anchored Identities and x402 Micropayments for AI Agents. <https://arxiv.org/abs/2507.19550>. arXiv:2507.19550 [cs.MA]. Accessed: 2026-05-31.
- [10] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. <https://arxiv.org/abs/2308.08155>. arXiv:2308.08155. Accessed: 2026-05-31.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. arXiv:2210.03629.